

ChaosJack

Giulia Bardi, Beatrice Pallotti, Elena Lungu, Evelyn Calandrini

Indice

1	Analisi	2
1.1	Descrizione e requisiti	2
1.2	Modello del Dominio	3
2	Design	5
2.1	Architettura	5
2.2	Design dettagliato	9
3	Sviluppo	17
3.1	Testing automatizzato	17
3.2	Note di sviluppo	18
3.2.1	Esempio	20
4	Commenti finali	22
4.1	Autovalutazione e lavori futuri	22
4.2	Difficoltà incontrate e commenti per i docenti	22
A	Guida utente	24
B	Esercitazioni di laboratorio	25
B.0.1	paolino.paperino@studio.unibo.it	25
B.0.2	paperon.depaperoni@studio.unibo.it	25

Capitolo 1

Analisi

1.1 Descrizione e requisiti

Il software ChaosJack va a fondere il gioco classico di Black Jack con le meccaniche strategiche e i modificatori tipici di giochi come Balatro, un incontro tra il Poker classico e un videogioco di strategia e "rompicapo". L'obiettivo principale del giocatore è quello di sfidare e vincere sia contro il banco sia contro gli altri giocatori presenti al tavolo. Oltre ai giocatori effettivi saranno presenti dei giocatori bot, che seguiranno le regole di gioco in automatico. Svolgimento della partita:

- Fase iniziale: Per partecipare, ogni giocatore effettua una puntata base. Il banco distribuisce due carte scoperte a ciascun giocatore e a se stesso (scartando la prima e mostrando la seconda).
- Valore delle carte: Le figure valgono 10, l'Asso vale 1 o 11 a discrezione del giocatore, le altre carte mantengono il loro valore nominale.
- Turno del giocatore: In base al proprio punteggio, il giocatore può chiedere ulteriori carte ("hit") più volte. Se la somma supera 21, il giocatore sballa ed è fuori dalla partita. Il turno termina quando il giocatore decide di fermarsi ("stand").
- Fase di puntata finale : Terminato il turno di tutti i giocatori, è possibile effettuare ulteriori scommesse scegliendo una somma o raddoppiando il precedente (questo solo se si posseggono due carte in mano).
- Turno del banco: Il banco pesca carte fermandosi obbligatoriamente al raggiungimento di un punteggio pari o superiori a 17. Se il banco supera 21, la vincita è contesa tra i giocatori ancora attivi con il punteggio più alto.

Condizioni di vittoria e punteggi : Essendo un tutti contro tutti, per vincere bisogna battere sia il banco che gli altri giocatori. A parità di punteggio, vince chi ha meno carte in mano. Se il pareggio persiste tra più giocatori, la vincita viene divisa. La vincita standard raddoppia il piatto. Il raggiungimento del Blackjack triplica il piatto, mentre una mano monocolora fornisce un moltiplicatore bonus. Inoltre, all'avvio di una partita, il giocatore potrebbe imbattersi in turni "speciali" e carte speciali con vincoli che alterano i punteggi.

Requisiti funzionali

- Il sistema deve permettere lo svolgimento della logica di base del Blackjack, includendo la distribuzione iniziale delle carte e il loro mescolamento. Inoltre deve garantire le azioni di richiesta delle carte, fermarsi, uscire e puntare.
- Il sistema deve calcolare correttamente i punteggi delle mani, gestendo la soglia massima del 21 e le regole specifiche associate alle singole carte.
- Il gioco deve applicare automaticamente le alterazioni delle regole durante i turni speciali.
- Il software deve gestire i portafogli virtuali (fish) sia del giocatore che del banco, aggiornandoli in base all'esito delle puntate.

Requisiti non funzionali

- L'interfaccia utente deve essere reattiva, restituendo un feedback visivo immediato a ogni azione del giocatore senza ritardi percettibili.
- Il sistema deve supportare l'aggiunta futura di nuovi modificatori, turni speciali o regole.

1.2 Modello del Dominio

L'ambiente principale è il tavolo, il quale riunisce tutte le entità coinvolte e scandisce lo svolgimento del gioco. Al tavolo siedono i partecipanti, ovvero il banco, i giocatori e i giocatori bot (i quali seguono logiche autonome). Ciascun partecipante è dotato di un proprio portafoglio di fiches e riceve una mano di carte. L'intero flusso della partita al tavolo è organizzato in una sequenza di fasi ben distinte (come la puntata iniziale o la fase di gioco). All'interno di ogni fase, l'azione si articola nei turni individuali dei partecipanti.

Durante le fasi di scommessa, le fiches convergono in un piatto centrale. Le carte vengono fornite da un mazzo, che include sia le carte classiche che speciali. Infine, specifiche partite possono essere alterate dall'applicazione di modificatori, i quali introducono vincoli temporanei alle regole di base alterando le normali fasi di gioco.

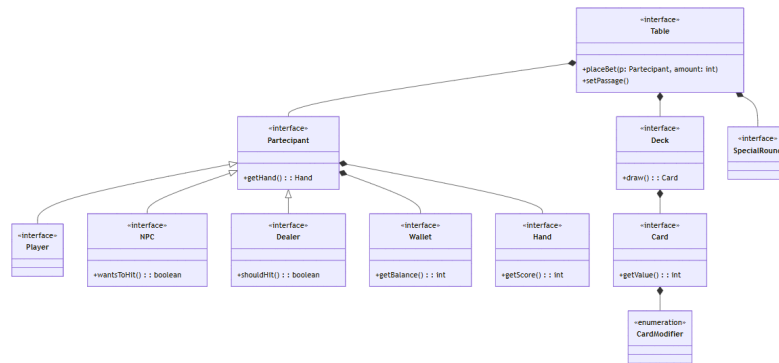


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali ed i rapporti fra loro

Capitolo 2

Design

2.1 Architettura

L'architettura adottata per il progetto ChaosJack segue il pattern architetturale MVC(Model-View-Controller). Questo pattern consente di avere una chiara separazione delle responsabilità e una maggiore flessibilità in fase di sviluppo, permettendoci di separare la logica di gioco dalla sua rappresentazione grafica. Ovvero è possibile sostituire il blocco della view senza richiedere alcuna modifica nel Controller e nel Model. L'architettura si basa su queste componenti principali:

- Model : GameEngine e Table
- Controller : GameFlowController e ActionController
- View : ViewManager

Il Model : Si occupa di gestire e mantenere in modo consistente lo stato interno del gioco (il mazzo di carte, i portafogli dei partecipanti, il calcolo dei punteggi nelle mani). Valida ed esegue tutte le mutazioni dello stato di gioco secondo le regole tradizionali del BlackJack e l'applicazione dei relativi modificatori speciali.

Il controller : Rappresenta il motore attivo che si occupa di dettare i tempi e l'ordine delle diverse fasi della partita (regolando i turni tra i giocatori presenti nel tavolo).

La View: Ha il ruolo generale di gestire la visualizzazione dei componenti all'interno della finestra, mostrando l'evoluzione della partita. Si occupa, inoltre, di gestire la fase di navigazione del menù, consentendo di avviare una nuova partita o vedere le statistiche dei round giocati. P

Grazie a questa architettura è possibile sostituire il blocco della view senza richiedere alcuna modifica nel Controller e nel Model. La View si limita a notificare le azioni dell'utente al Controller. Sarà poi il Controller a invocare i metodi appropriati sul Model per aggiornare lo stato.

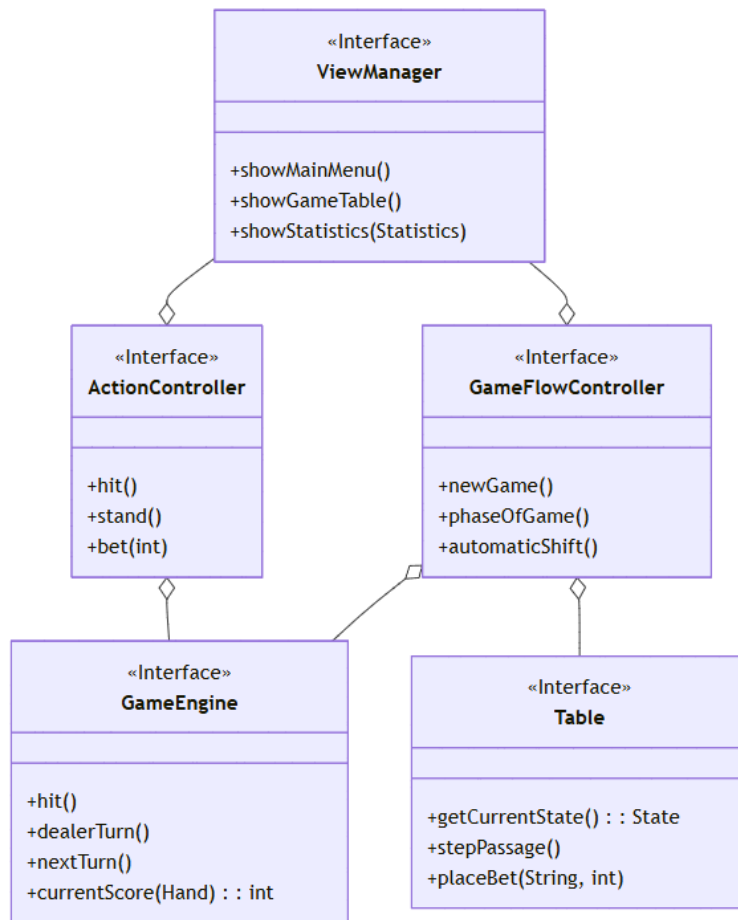


Figura 2.1: Schema UML dell'architettura MVC realizzata per ChaosJack

Elementi positivi

- Si mostrano pochi, mirati schemi UML dai quali si deduce con chiarezza quali sono le parti principali del software e come interagiscono fra loro.

- Si mette in evidenza se e come il pattern architetturale model-view-controller è stato applicato, anche con l'uso di un UML che mostri le interfacce principali ed i rapporti fra loro.
- Si discute se sia semplice o meno, con l'architettura scelta, sostituire in blocco la view: in un MVC ben fatto, controller e modello non dovrebbero in alcun modo cambiare se si transitasse da una libreria grafica ad un'altra (ad esempio, da Swing a JavaFX, o viceversa).

Elementi negativi

- L'architettura è fatta in modo che sia impossibile riusare il modello per un software diverso che affronta lo stesso problema.
- L'architettura è tale che l'aggiunta di una funzionalità sul controller impatta pesantemente su view e/o modello.
- L'architettura è tale che la sostituzione in blocco della view impatta sul controller o, peggio ancora, sul modello.
- Si presentano UML caotici, difficili da leggere.
- Si presentano UML in cui sono mostrati elementi di dettaglio non appartenenti all'architettura, ad esempio includenti campi o con metodi che non interessano la parte di interazione fra le componenti principali del software.
- Si presentano schemi UML con classi (nel senso UML del termine) che “galleggiano” nello schema, non connesse, ossia senza relazioni con il resto degli elementi inseriti.
- Si presentano elementi di design di dettaglio, ad esempio tutte le classi e interfacce del modello o della view.
- Si discutono aspetti implementativi, ad esempio eventuali librerie usate oppure dettagli di codice.

Esempio

L'architettura di GLaDOS segue il pattern architetturale MVC. Più nello specifico, a livello architetturale, si è scelto di utilizzare MVC in forma “ECB”, ossia “entity-control-boundary”¹. GLaDOS implementa l'interfaccia AI, ed

¹Si fa presente che il pattern ECB effettivamente esiste in letteratura come “istanza” di MVC, e chi volesse può utilizzarlo come reificazione di MVC.

è il controller del sistema. Essendo una intelligenza artificiale, è una classe attiva. GLaDOS accetta la registrazione di Input ed Output, che fanno parte della “view” di MVC, e sono il “boundary” di ECB. Gli Input rappresentano delle nuove informazioni che vengono fornite all’IA, ad esempio delle modifiche nel valore di un sensore, oppure un comando da parte dell’operatore. Questi input infatti forniscono eventi. Ottenere un evento è un’operazione bloccante: chi la esegue resta in attesa di un effettivo evento. Di fatto, quindi, GLaDOS si configura come entità *reattiva*. Ogni volta che c’è un cambio alla situazione del soggetto, GLaDOS notifica i suoi Output, informandoli su quale sia la situazione corrente. Conseguentemente, GLaDOS è un “observable” per Output.

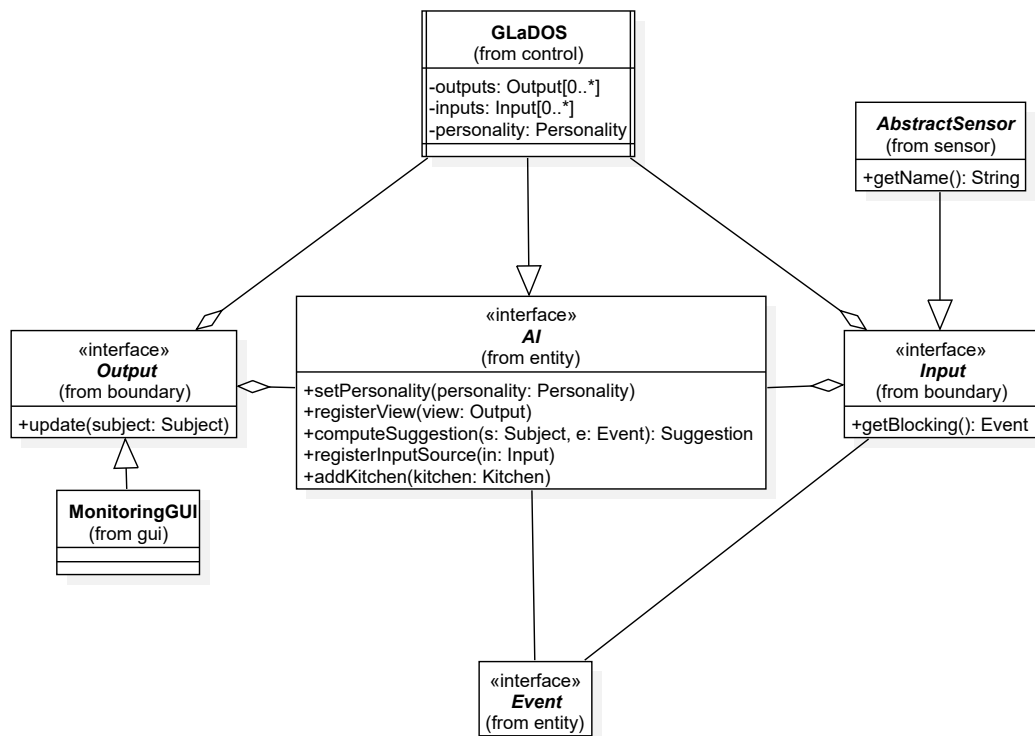


Figura 2.2: Schema UML architetturale di GLaDOS. L’interfaccia **GLaDOS** è il controller del sistema, mentre **Input** ed **Output** sono le interfacce che mappano la view (o, più correttamente in questo specifico esempio, il boundary). Un’eventuale interfaccia grafica interattiva dovrà implementarle entrambe.

Con questa architettura, possono essere aggiunti un numero arbitrario di input ed output all’intelligenza artificiale. Ovviamente, mentre l’aggiunta di output è semplice e non richiede alcuna modifica all’IA, la presenza di nuovi tipi di evento richiede invece in potenza aggiunte o rifiniture a GLaDOS.

Questo è dovuto al fatto che nuovi Input rappresentano di fatto nuovi elementi della business logic, la cui alterazione od espansione inevitabilmente impatta il controller del progetto.

In Figura 2.2 è esemplificato il diagramma UML architetturale.

2.2 Design dettagliato

In questa sezione si possono approfondire alcuni elementi del design con maggior dettaglio. Mentre ci attendiamo principalmente (o solo) interfacce negli schemi UML delle sezioni precedenti, in questa sezione è necessario scendere in maggior dettaglio presentando la struttura di alcune sottoparti rilevanti dell'applicazione. È molto importante che, descrivendo la soluzione ad un problema, quando possibile si mostri che non si è re-inventata la ruota ma si è applicato un design pattern noto. Che si sia utilizzato (o riconosciuto) o meno un pattern noto, è comunque bene definire qual è il problema che si è affrontato, qual è la soluzione messa in campo, e quali motivazioni l'hanno spinta. È assolutamente inutile, ed è anzi controproducente, descrivere classe-per-classe (o peggio ancora metodo-per-metodo) com'è fatto il vostro software: è un livello di dettaglio proprio della documentazione dell'API (deducibile dalla Javadoc).

È necessario che ciascun membro del gruppo abbia una propria sezione di design dettagliato, di cui sarà il solo responsabile. Ciascun autore dovrà spiegare in modo corretto e giustamente approfondito (non troppo in dettaglio, non superficialmente) il proprio contributo. È importante focalizzarsi sulle scelte che hanno un impatto positivo sul riuso, sull'estensibilità, e sulla chiarezza dell'applicazione. Esattamente come nessun ingegnere meccanico presenta un solo foglio con l'intero progetto di una vettura di Formula 1, ma molteplici fogli di progetto che mostrano a livelli di dettaglio differenti le varie parti della vettura e le modalità di connessione fra le parti, così ci aspettiamo che voi, futuri ingegneri informatici, ci presentiate prima una visione globale del progetto, e via via siate in grado di dettagliare le singole parti, scartando i componenti che non interessano quella in esame. Per continuare il parallelo con la vettura di Formula 1, se nei fogli di progetto che mostrano il design delle sospensioni anteriori appaiono pezzi che appartengono al volante o al turbo, c'è una chiara indicazione di qualche problema di design.

Si divida la sezione in sottosezioni, e per ogni aspetto di design che si vuole approfondire, si presenti:

1. : una breve descrizione in linguaggio naturale del problema che si vuole risolvere, se necessario ci si può aiutare con schemi o immagini;

2. : una descrizione della soluzione proposta, analizzando eventuali alternative che sono state prese in considerazione, e che descriva pro e contro della scelta fatta;
3. : uno schema UML che aiuti a comprendere la soluzione sopra descritta;
4. : se la soluzione è stata realizzata utilizzando uno o più pattern noti, si spieghi come questi sono reificati nel progetto (ad esempio: nel caso di Template Method, qual è il metodo template; nel caso di Strategy, quale interfaccia del progetto rappresenta la strategia, e quali sono le sue implementazioni; nel caso di Decorator, qual è la classe astratta che fa da Decorator e quali sono le sue implementazioni concrete; eccetera);

La presenza di pattern di progettazione *correttamente utilizzati* è valutata molto positivamente. L'uso inappropriato è invece valutato negativamente: a tal proposito, si raccomanda di porre particolare attenzione all'abuso di Singleton, che, se usato in modo inappropriato, è di fatto un anti-pattern.

Elementi positivi

- Ogni membro del gruppo discute le proprie decisioni di progettazione, ed in particolare le azioni volte ad anticipare possibili cambiamenti futuri (ad esempio l'aggiunta di una nuova funzionalità, o il miglioramento di una esistente).
- Si mostrano le principali interazioni fra le varie componenti che collaborano alla soluzione di un determinato problema.
- Si identificano, utilizzano *appropriatamente*, e descrivono diversi design pattern.
- Ogni membro del gruppo identifica i pattern utilizzati nella sua sottoparte.
- Si mostrano gli aspetti di design più rilevanti dell'applicazione, mettendo in luce la maniera in cui si è costruita la soluzione ai problemi descritti nell'analisi.
- Si tralasciano aspetti strettamente implementativi e quelli non rilevanti, non mostrandoli negli schemi UML (ad esempio, campi privati) e non descrivendoli.

- Ciascun elemento di design identificato presenta una piccola descrizione del problema calato nell'applicazione, uno schema UML che ne mostra la concretizzazione nelle classi del progetto, ed una breve descrizione della motivazione per cui tale soluzione è stata scelta, specialmente se è stato utilizzato un pattern noto. Ad esempio, se si dichiara di aver usato Observer, è necessario specificare chi sia l'observable e chi l'observer; se si usa Template Method, è necessario indicare quale sia il metodo template; se si usa Strategy, è necessario identificare l'interfaccia che rappresenta la strategia; e via dicendo.

Elementi negativi

- Il design del modello risulta scorrelato dal problema descritto in analisi.
- Si tratta in modo prolisso, classe per classe, il software realizzato, o comunque si riduce la sezione ad un mero elenco di quanto fatto.
- Non si presentano schemi UML esemplificativi.
- Non si individuano design pattern, o si individuano in modo errato (si spaccia per design pattern qualcosa che non lo è).
- Si utilizzano design pattern in modo inopportuno. Un esempio classico è l'abuso di Singleton per entità che possono essere univoche ma non devono necessariamente esserlo. Si rammenta che Singleton ha senso nel secondo caso (ad esempio **System** e **Runtime** sono singleton), mentre rischia di essere un problema nel secondo. Ad esempio, se si rendesse singleton il motore di un videogioco, sarebbe impossibile riusarlo per costruire un server per partite online (dove, presumibilmente, si gestiscono parallelamente più partite).
- Si producono schemi UML caotici e difficili da leggere, che comprendono inutili elementi di dettaglio.
- Si presentano schemi UML con classi (nel senso UML del termine) che “galleggiano” nello schema, non connesse, ossia senza relazioni con il resto degli elementi inseriti.
- Si tratta in modo inutilmente prolisso la divisione in package, elencando ad esempio le classi una per una.

Esempio minimale (e quindi parziale) di sezione di progetto con UML ben realizzati

Personalità intercambiabili

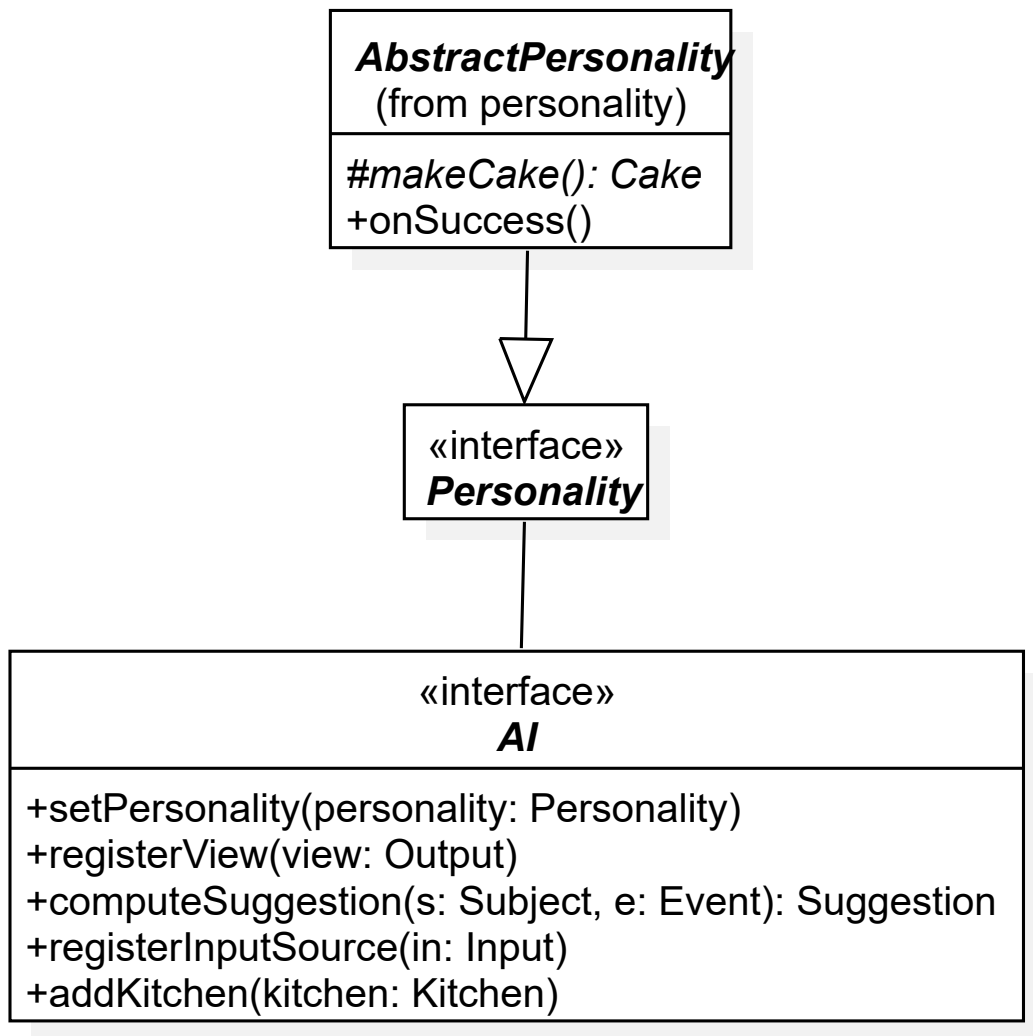


Figura 2.3: Rappresentazione UML del pattern Strategy per la personalità di GLaDOS

Problema GLaDOS ha più personalità intercambiabili, la cui presenza deve essere trasparente al client.

Soluzione Il sistema per la gestione della personalità utilizza il *pattern Strategy*, come da Figura 2.3: le implementazioni di **Personality** possono

essere modificate, e la modifica impatta direttamente sul comportamento di GLaDOS.

Riuso del codice delle personalità

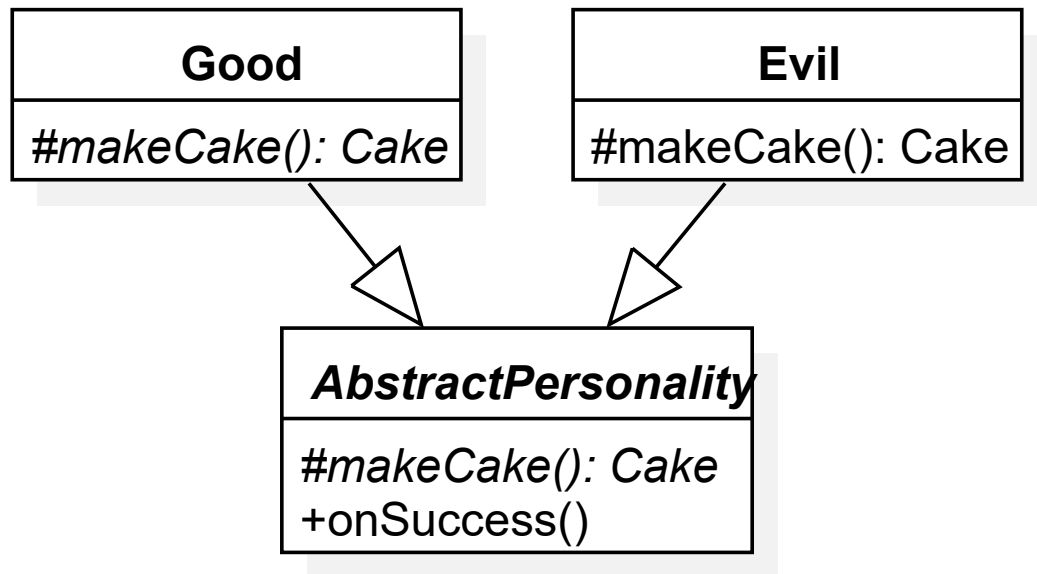


Figura 2.4: Rappresentazione UML dell'applicazione del pattern Template Method alla gerarchia delle Personalità

Problema In fase di sviluppo, sono state sviluppate due personalità, una buona ed una cattiva. Quella buona restituisce sempre una torta vera, mentre quella cattiva restituisce sempre la promessa di una torta che verrà in realtà disattesa. Ci si è accorti che diverse personalità condividevano molto del comportamento, portando a classi molto simili e a duplicazione.

Soluzione Dato che le due personalità differiscono solo per il comportamento da effettuarsi in caso di percorso completato con successo, è stato utilizzato il *pattern template method* per massimizzare il riuso, come da Figura 2.4. Il metodo template è `onSuccess()`, che chiama un metodo astratto e protetto `makeCake()`.

Gestione di output multipli

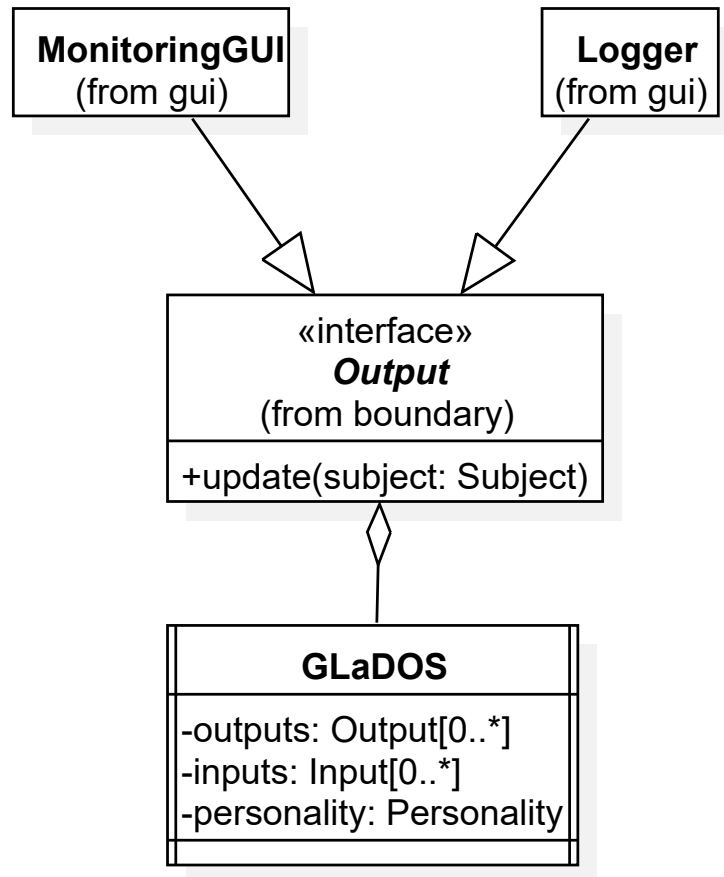


Figura 2.5: Il pattern Observer è usato per consentire a GLaDOS di informare tutti i sistemi di output in ascolto

Problema Il sistema deve supportare output multipli. In particolare, si richiede che vi sia un logger che stampa a terminale o su file, e un'interfaccia grafica che mostri una rappresentazione grafica del sistema.

Soluzione Dato che i due sistemi di reporting utilizzano le medesime informazioni, si è deciso di raggrupparli dietro l'interfaccia **Output**. A questo punto, le due possibilità erano quelle di far sì che **GLaDOS** potesse pilotarle entrambe. Invece di fare un sistema in cui questi output sono obbligatori e connessi, si è deciso di usare maggior flessibilità (anche in vista di future estensioni) e di adottare una comunicazione uno-a-molti fra **GLaDOS** ed i sistemi di output. La scelta è quindi ricaduta sul *pattern Observer*: **GLaDOS** è

observable, e le istanze di `Output` sono observer. Il suo utilizzo è esemplificato in Figura 2.5

Contro-esempio: pessimo diagramma UML

In Figura 2.6 è mostrato il modo **sbagliato** di fare le cose. Questo schema è fatto male perché:

- È caotico.
- È difficile da leggere e capire.
- Vi sono troppe classi, e non si capisce bene quali siano i rapporti che intercorrono fra loro.
- Si mostrano elementi implementativi irrilevanti, come i campi e i metodi privati nella classe `AbstractEnvironment`.
- Se l'intenzione era quella di costruire un diagramma architetturale, allora lo schema è ancora più sbagliato, perché mostra pezzi di implementazione.
- Una delle classi, in alto al centro, galleggia nello schema, non connessa a nessuna altra classe, e di fatto costituisce da sola un secondo schema UML scorrelato al resto
- Le interfacce presentano tutti i metodi e non una selezione che aiuti il lettore a capire quale parte del sistema si vuol mostrare.

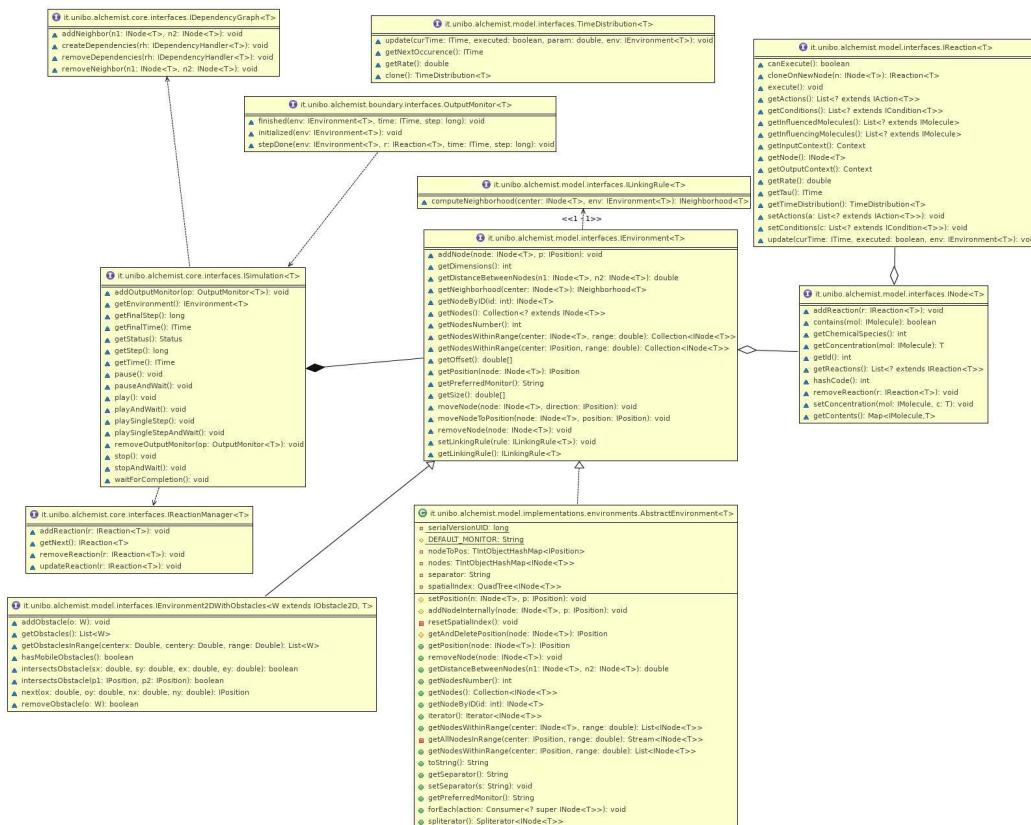


Figura 2.6: Schema UML mal fatto e con una pessima descrizione, che non aiuta a capire. Don't try this at home.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Il testing automatizzato è un requisito di qualunque progetto software che si rispetti, e consente di verificare che non vi siano regressioni nelle funzionalità a fronte di aggiornamenti. Per quanto riguarda questo progetto è considerato sufficiente un test minimale, a patto che sia completamente automatico. Test che richiedono l'intervento da parte dell'utente sono considerati *negativamente* nel computo del punteggio finale.

Elementi positivi

- Si descrivono molto brevemente i componenti che si è deciso di sottoporre a test automatizzato.
- Si utilizzano suite specifiche (e.g. JUnit) per il testing automatico.

Elementi negativi

- Non si realizza alcun test automatico.
- La non presenza di testing viene aggravata dall'adduzione di motivazioni non valide. Ad esempio, si scrive che l'interfaccia grafica non è testata automaticamente perché è *impossibile* farlo¹.
- Si descrive un testing di tipo manuale in maniera prolissa.

¹Testare in modo automatico le interfacce grafiche è possibile (si veda, come esempio, <https://github.com/TestFX/TestFX>), semplicemente nel corso non c'è modo e tempo di introdurre questo livello di complessità. Il fatto che non vi sia stato insegnato come farlo non implica che sia impossibile!

- Si descrivono test effettuati manualmente che sarebbero potuti essere automatizzati, ad esempio scrivendo che si è usata l'applicazione manualmente.
- Si descrivono test non presenti nei sorgenti del progetto.
- I test, quando eseguiti, falliscono.

3.2 Note di sviluppo

Questa sezione, come quella riguardante il design dettagliato va svolta **singolarmente da ogni membro del gruppo**. Nella prima parte, ciascuno dovrà mostrare degli esempi di codice particolarmente ben realizzati, che dimostrino proefficienza con funzionalità avanzate del linguaggio e capacità di spingersi oltre le librerie mostrate a lezione.

- **Elencare** (fare un semplice elenco per punti, non un testo!) le feature *avanzate* del linguaggio e dell'ecosistema Java che sono state utilizzate. Le feature di interesse sono:
 - Progettazione con generici, ad esempio costruzione di nuovi tipi generici, e uso di generici bounded. L'uso di classi generiche di libreria non è considerato avanzato.
 - Uso di lambda expressions
 - Uso di **Stream**, di **Optional** o di altri costrutti funzionali
 - Uso di reflection
 - Definizione ed uso di nuove annotazioni
 - Uso del Java Platform Module System
 - Uso di parti della libreria JDK non spiegate a lezione (networking, compressione, parsing XML, eccetera...)
 - Uso di librerie di terze parti (incluso JavaFX): Google Guava, Apache Commons...
- Si faccia molta attenzione a non scrivere banalità, elencando qui features di tipo “core”, come le eccezioni, le enumerazioni, o le inner class: nessuna di queste è considerata avanzata.
- Per ogni feature avanzata, mostrata, includere:
 - Nome della feature

– Permalink GitHub al punto nel codice in cui è stata utilizzata

In questa sezione, *dopo l'elenco*, vanno menzionati ed attribuiti con precisione eventuali pezzi di codice “riadattati” (o scopiazzati...) da Internet o da altri progetti, pratica che tolleriamo ma che non raccomandiamo. Si rammenta agli studenti che non è consentito partire da progetti esistenti e procedere per modifiche successive. Si ricorda anche che i docenti hanno in mano strumenti antiplagio piuttosto raffinati e che “capiscono” il codice e la storia delle modifiche del progetto, per cui tecniche banali come cambiare nomi (di classi, metodi, campi, parametri, o variabili locali), aggiungere o togliere commenti, oppure riordinare i membri di una classe vengono individuate senza problemi. Le regole del progetto spiegano in dettaglio l’approccio dei docenti verso atti gravi come il plagiarismo.

I pattern di design **non** vanno messi qui. L’uso di pattern di design (come suggerisce il nome) è un aspetto avanzato di design, non di implementazione, e non va in questa sezione.

Elementi positivi

- Si elencano gli aspetti avanzati di linguaggio che sono stati impiegati
- Si elencano le librerie che sono state utilizzate
- Per ciascun elemento, si fornisce un permalink
- Ogni permalink fa riferimento ad uno snippet di codice scritto dall’autore della sezione (i docenti verificheranno usando `git blame`)
- Se si è utilizzato un particolare algoritmo, se ne cita la fonte originale. Ad esempio, se si è usato Mersenne Twister per la generazione di numeri pseudo-random, si cita [MN98].
- Si identificano parti di codice prese da altri progetti, dal web, o comunque scritte in forma originale da altre persone. In tal senso, si ricorda che agli ingegneri non è richiesto di re-inventare la ruota continuamente: se si cita debitamente la sorgente è tollerato fare uso di snippet di codice open source per risolvere velocemente problemi non banali. Nel caso in cui si usino snippet di codice di qualità discutibile, oltre a menzionarne l’autore originale si invitano gli studenti ad adeguare tali parti di codice agli standard e allo stile del progetto. Contestualmente, si fa presente che è largamente meglio fare uso di una libreria che copiarsi pezzi di codice: qualora vi sia scelta (e tipicamente c’è), si preferisca la prima via.

Elementi negativi

- Si elencano feature core del linguaggio invece di quelle segnalate. Esempi di feature core da non menzionare sono:
 - eccezioni;
 - classi innestate;
 - enumerazioni;
 - interfacce.
- Si elencano applicazioni di terze parti (peggio se per usarle occorre licenza, e lo studente ne è sprovvisto) che non c'entrano nulla con lo sviluppo, ad esempio:
 - Editor di grafica vettoriale come Inkscape o Adobe Illustrator;
 - Editor di grafica scalare come GIMP o Adobe Photoshop;
 - Editor di audio come Audacity;
 - Strumenti di design dell'interfaccia grafica come SceneBuilder: il codice è in ogni caso inteso come sviluppato da voi.
- Si descrivono aspetti di scarsa rilevanza, o si scende in dettagli inutili.
- Sono presenti parti di codice sviluppate originalmente da altri che non vengono debitamente segnalate. In tal senso, si ricorda agli studenti che i docenti hanno accesso a tutti i progetti degli anni passati, a Stack Overflow, ai principali blog di sviluppatori ed esperti Java, ai blog dedicati allo sviluppo di soluzioni e applicazioni (inclusi blog dedicati ad Android e allo sviluppo di videogame), nonché ai vari GitHub, GitLab, e Bitbucket. Conseguentemente, è *molto* conveniente *citare* una fonte ed usarla invece di tentare di spacciare per proprio il lavoro di altri.
- Si elencano design pattern

3.2.1 Esempio

Utilizzo della libreria SLF4J

Utilizzata in vari punti. Un esempio è <https://github.com/AlchemistSimulator/Alchemist/blob/5c17f8b76920c78d955d478864ac1f11508ed9ad/alchemist-swingui/src/main/java/it/unibo/alchemist/boundary/swingui/effect/impl/EffectBuilder.java#L49>

Utilizzo di LoadingCache dalla libreria Google Guava

Permalink: <https://github.com/AlchemistSimulator/Alchemist/blob/d8a1799027d7d685569e15316a32e6394632ce71/alchemist-incarnation-protelis/src/main/java/it/unibo/alchemist/protelis/AlchemistExecutionContext.java#L141-L143>

Utilizzo di Stream e lambda expressions

Usate pervasivamente. Il seguente è un singolo esempio. Permalink: <https://github.com/AlchemistSimulator/Alchemist/blob/d8a1799027d7d685569e15316a32e6394632ce71/alchemist-incarnation-protelis/src/main/java/it/unibo/alchemist/model/ProtelisIncarnation.java#L98-L120>

Scrittura di metodo generico con parametri contravarianti

Permalink: <https://github.com/AlchemistSimulator/Alchemist/blob/d8a1799027d7d685569e15316a32e6394632ce71/alchemist-incarnation-protelis/src/main/java/it/unibo/alchemist/protelis/AlchemistExecutionContext.java#L141-L143>

Protezione da corse critiche usando Semaphore

Permalink: <https://github.com/AlchemistSimulator/Alchemist/blob/d8a1799027d7d685569e15316a32e6394632ce71/alchemist-incarnation-protelis/src/main/java/it/unibo/alchemist/model/ProtelisIncarnation.java#L388-L440>

Capitolo 4

Commenti finali

In quest'ultimo capitolo si tirano le somme del lavoro svolto e si delineano eventuali sviluppi futuri.

Nessuna delle informazioni incluse in questo capitolo verrà utilizzata per formulare la valutazione finale, a meno che non sia assente o manchino delle sezioni obbligatorie. Al fine di evitare pregiudizi involontari, l'intero capitolo verrà letto dai docenti solo dopo aver formulato la valutazione.

4.1 Autovalutazione e lavori futuri

È richiesta una sezione per ciascun membro del gruppo, obbligatoriamente. Ciascuno dovrà autovalutare il proprio lavoro, elencando i punti di forza e di debolezza in quanto prodotto. Si dovrà anche cercare di descrivere *in modo quanto più obiettivo possibile* il proprio ruolo all'interno del gruppo. Si ricorda, a tal proposito, che ciascuno studente è responsabile solo della propria sezione: non è un problema se ci sono opinioni contrastanti, a patto che rispecchino effettivamente l'opinione di chi le scrive. Nel caso in cui si pensasse di portare avanti il progetto, ad esempio perché effettivamente impiegato, o perché sufficientemente ben riuscito da poter esser usato come dimostrazione di esser capaci progettisti, si descriva brevemente verso che direzione portarlo.

4.2 Difficoltà incontrate e commenti per i docenti

Questa sezione, **opzionale**, può essere utilizzata per segnalare ai docenti eventuali problemi o difficoltà incontrate nel corso o nello svolgimento del

progetto, può essere vista come una seconda possibilità di valutare il corso (dopo quella offerta dalle rilevazioni della didattica) avendo anche conoscenza delle modalità e delle difficoltà collegate all'esame, cosa impossibile da fare usando le valutazioni in aula per ovvie ragioni. È possibile che alcuni dei commenti forniti vengano utilizzati per migliorare il corso in futuro: sebbene non andrà a vostro beneficio, potreste fare un favore ai vostri futuri colleghi. Ovviamente *il contenuto della sezione non impatterà il voto finale*.

Appendice A

Guida utente

Capitolo in cui si spiega come utilizzare il software. Nel caso in cui il suo uso sia del tutto banale, tale capitolo può essere omesso. A tal riguardo, si fa presente agli studenti che i docenti non hanno mai utilizzato il software prima, per cui aspetti che sembrano del tutto banali a chi ha sviluppato l'applicazione possono non esserlo per chi la usa per la prima volta. Se, ad esempio, per cominciare una partita con un videogioco è necessario premere la barra spaziatrice, o il tasto “P”, è necessario che gli studenti lo segnalino.

Elementi positivi

- Si istruisce in modo semplice l'utente sull'uso dell'applicazione, eventualmente facendo uso di schermate e descrizioni.

Elementi negativi

- Si descrivono in modo eccessivamente minuzioso tutte le caratteristiche, anche minori, del software in oggetto.
- Manca una descrizione che consenta ad un utente qualunque di utilizzare almeno le funzionalità primarie dell'applicativo.

Appendice B

Esercitazioni di laboratorio

In questo capitolo ciascuno studente elenca gli esercizi di laboratorio che ha svolto (se ne ha svolti), elencando i permalink dei post sul forum dove è avvenuta la consegna. Questa sezione potrebbe essere processata da strumenti automatici, per cui link a oggetti diversi dal permalink della consegna, errori nell'email o nel nome del laboratorio possono portare ad ignorare alcune consegne, si raccomanda la massima precisione.

Esempio

B.0.1 paolino.paperino@studio.unibo.it

- Laboratorio 04: <https://virtuale.unibo.it/mod/forum/discuss.php?d=12345#p123456>
- Laboratorio 06: <https://virtuale.unibo.it/mod/forum/discuss.php?d=22222#p222222>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=99999#p999999>

B.0.2 paperon.depaperoni@studio.unibo.it

- Laboratorio 04: <https://virtuale.unibo.it/mod/forum/discuss.php?d=12345#p123456>
- Laboratorio 05: <https://virtuale.unibo.it/mod/forum/discuss.php?d=22222#p222222>

- Laboratorio 06: <https://virtuale.unibo.it/mod/forum/discuss.php?d=999999#p999999>
- Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=222222#p222222>
- Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=999999#p999999>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=222222#p222222>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=999999#p999999>
- Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=222222#p222222>

Bibliografia

- [MN98] Makoto Matsumoto and Takuji Nishimura. Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Trans. Model. Comput. Simul.*, 8(1):3–30, January 1998.